

Top 40 One-Page Cheat Sheet

Last-hour revision for the 40 questions in `top-40.md`. Each line is the *kernel* of the answer — the sentence you must be able to say before you add detail. If a line does not immediately unfold into a two-minute answer, reread that question in the book.

If you only have an hour, drill these ten: Q004, Q051, Q085, Q112, Q114, Q121, Q137, Q143, Q173, Q197.

Core Java

Q004 equals/hashCode — Equal objects must have equal hash codes. Mutating a field used in the hash after insertion loses the object in its old bucket; prefer immutable business keys, never a JPA id assigned at flush.

Q008 checked vs unchecked — Checked means the caller must decide and it is part of the signature; unchecked for programming errors and faults nobody can handle locally. Translate at the boundary, never swallow.

Q012 immutability — Final class, private final fields set in the constructor, defensive copies in and out, no setters. Safe to share across threads with no synchronisation.

Q019 dates and time zones — Store and transmit UTC (Instant, timestampz), ISO-8601 with offset; apply a zone only at the edge; inject a Clock so time is testable.

Q020 money — BigDecimal with explicit scale and RoundingMode, built from a String, compared with compareTo. Binary floating point cannot represent 0.1, so double drifts.

Collections and streams

Q031 List, Set or Map — Choose on semantics first: duplicates allowed, order significant, lookup by key. Size and access pattern come second.

Q032 HashMap internals — Power-of-two buckets, hash spreading, a bin becomes a tree at 8 entries once capacity is 64, resize at load factor 0.75. A poor hash costs performance, not correctness.

Q038 grouping and aggregating — filter and map, then a collector: groupingBy with a downstream such as summingLong or mapping. toMap throws on duplicate keys and rejects null values.

Q041 parallel streams — Only for CPU-bound, large, splittable, side-effect-free work, and never for blocking I/O; they share the common ForkJoinPool with the rest of the application.

Concurrency and JVM

Q051 races, visibility, atomicity — Visibility is whether another thread sees the write; atomicity is whether the operation is indivisible. count++ needs both, and happens-before is what makes the outcome defined.

Q053 volatile — Enough for a status flag or safe publication of an immutable reference; not enough for read-modify-write, which needs an Atomic type or a lock.

Q055 executor sizing — CPU-bound near the core count, I/O-bound higher but capped by what the downstream can take. Always a bounded queue with an explicit rejection policy; unbounded queues hide overload until memory runs out.

Q058 deadlock — Four conditions must hold; break circular wait with a global lock order or tryLock with a timeout. jcmd <pid> Thread.print names the deadlock directly.

Q064 virtual threads — Cheap blocking for I/O-bound work, no help for CPU-bound. The two traps are pooling inside synchronized and unbounded concurrency hitting a fixed-size pool.

Spring

Q076 constructor injection — Gives final fields, fail-fast startup and plain construction in tests; field injection hides dependencies and lets circular graphs survive.

Q080 auto-configuration — Classes listed in AutoConfiguration.imports, filtered by @Conditional checks; your own bean wins through @ConditionalOnMissingBean. Diagnose with the condition evaluation report.

Q082 MVC lifecycle — Filters, then DispatcherServlet, handler mapping, argument resolution and validation, the handler, then a return-value handler and message converter; exceptions go to the resolver chain.

Q085 @Transactional proxies — Proxy-based, so self-invocation and private methods bypass it. Runtime exceptions roll back, checked ones do not unless you declare rollbackFor.

Q086 REQUIRED vs REQUIRES_NEW — REQUIRED joins the caller's transaction; REQUIRES_NEW suspends it and takes a second connection, which can deadlock against locks the outer transaction still holds.

Q099 pool sizing — Small pools usually beat large ones, and the total across instances must fit the database limit. Connection-timeout errors mean queuing for a connection, not necessarily a slow database.

SQL, JPA and Hibernate

Q111 B-tree indexes — Composite indexes are usable left to right; a function on the column, a type mismatch or low selectivity defeats them. Every index is a write cost.

Q112 EXPLAIN ANALYZE — Compare estimated with actual rows: a large gap means stale statistics. Read the slowest node rather than the plan's shape, and add BUFFERS for real I/O.

Q114 isolation levels — Dirty, non-repeatable and phantom reads. PostgreSQL's Read Committed stops only dirty reads; Repeatable Read still allows write skew, and Serializable requires retrying 40001 errors.

Q116 optimistic vs pessimistic — @Version for rare conflicts plus a retry path; PESSIMISTIC_WRITE for hot rows, kept short and always acquired in the same order.

Q121 N+1 — Detect by counting statements, not by reading code. Fix with JOIN FETCH or @EntityGraph, and watch pagination with collection joins; open-in-view=true hides the problem until production.

Q126 external calls in transactions — Do not hold a transaction open across an HTTP call: it pins a connection for the remote timeout. Commit first, then publish, using the outbox or an after-commit event.

HTTP, REST and microservices

Q137 status codes — 200, 201 with Location, 202 for accepted work, 204 for no body; 400 or 422 for validation, 401 versus 403, 404 to hide existence, 409 for conflict, 412 for preconditions, 429 with Retry-After.

Q141 idempotency — A client-generated key per logical operation, stored under a unique constraint: same key and payload replays the stored response, same key with a different payload is a conflict.

Q143 timeouts and retries — Every timeout shorter than the caller's deadline; retry only idempotent, retryable failures with capped exponential backoff plus jitter, inside a total budget and behind a circuit breaker.

Q150 order workflow — Idempotency key, validate, persist a pending order and an outbox row in one transaction, publish after commit, drive a state machine forward with compensations for failures.

Q152 duplicate requests — A unique constraint is the durable guarantee, because check-then-insert races. Catch the violation and treat it as the duplicate it is.

Security

Q162 sessions vs JWT — Sessions revoke instantly; JWTs scale statelessly but cannot be un-issued, so use a short access token with a rotating refresh token and keep neither in localStorage.

Q165 CORS vs CSRF — CORS relaxes the browser's same-origin policy and protects nothing server-side; CSRF matters only when credentials are ambient, such as cookies, not for bearer headers.

Q168 resource-level authorization — Authorise the record, not just the route: scope every query by owner or tenant, and return 404 instead of 403 when existence itself is sensitive.

Testing and debugging

Q173 test layers — Unit for logic, slice for wiring, Testcontainers integration for real SQL behaviour, a few end-to-end. Fast and deterministic beats exhaustive.

Q182 slow endpoint — Work outside in: percentiles, then the trace breakdown, then SQL and pool waits, then a thread dump. Measure before changing anything; p99 is not the average.

Messaging and caching

Q187 at-least-once — Consumers must be idempotent: a processed-event table keyed by event id, acknowledgement after the business commit, and a dead-letter topic for poison messages.

Q189 transactional outbox — Write the state change and the event in one local transaction and publish from the outbox after commit, turning a dual write into one atomic write plus retryable delivery.

Production

Q197 observability — Metrics say something is wrong, traces say where, logs say why, and one correlation id ties them together. Guard metric cardinality.

Q200 production triage — Define the failure and its blast radius, look for the first bad deploy or config change, follow a single correlation id end to end, mitigate before fixing.